

Schulprojekt · Softwareentwicklung

Wohnungen finden. Bezirk auswählen.

TopImmo macht geladene Mietwohnungsangebote in Stuttgart über eine Karte und eine Angebotsliste zugänglich.



Gemeinsam begonnen. Inzwischen allein weitergeführt. Aktueller Stand und offene Punkte werden getrennt dargestellt.

Ein Produkt mit API, Datenbank und C#-Backend.

Technische Anforderungen und nachvollziehbare Entwicklung gehören zusammen.

C#

Backend

ASP.NET Core mit Controllern, Services und Dependency Injection.

API

Externe Angebote

ImmoScout24 über RapidAPI; Verarbeitung erfolgt im Backend.

SQLite

Persistente Accounts

Benutzer und Refresh-Tokens werden mit EF Core gespeichert.

10 %

Scrum

40 %

Implementierung

10 %

Präsentation

40 %

Individuelle Leistung

Zusätzlich gefordert: Scrum-Nachweise, Architektur, wichtige Algorithmen und lauffähige Abgabe mit Startanleitung. Gewichtung laut Bewertung.xlsx.

Zwei Datenwege. Ein gemeinsames Backend.

Angular im Browser

Login/Register · Karte/Liste/Details · HttpClient-Services · Guard + Interceptor

↓ HTTP / JSON · Bearer-Token für Immobilienanfragen

ASP.NET Core / C#

Controller · Services · serverseitige Authentifizierung

AuthController → AuthService

BCrypt-Passwortprüfung · TokenService · EF Core

↓ SQLite: Users + RefreshTokens

RealEstateController [Authorize]

DistrictDataService · Filter/Zahlen · RAM-Cache 5 min

↔ ImmoScoutAPIService → RapidAPI

SQLite persistiert Konten und Tokens. Wohnungsangebote liegen derzeit im RAM-Cache. Der Server prüft den Zugriff; der Frontend-Guard unterstützt die Navigation.

Architekturdokument · Code: Program.cs · Z. 14/20 · ApplicationDbContext · Z. 11 · RealEstateController · Z. 8

Aus Angeboten werden Bezirke und Trefferzahlen.

1

Laden

Cache prüfen; bei Bedarf API-JSON lesen.

2

Miettyp filtern

Nur `apartmentrent` behalten.

3

Bezirk ableiten

Adresszeile auswerten und Stuttgart-Präfix normalisieren.

4

Zählen + sortieren

Nach Bezirk gruppieren; Anzahl absteigend sortieren.

5

Auswahl filtern

Gewählten Bezirk auf die Angebotsliste anwenden.

6

JSON darstellen

Liste, Bezirkszahlen und SVG-Zuordnung ans Frontend.

Cache: 5 Minuten. Eine Ladesperre vermeidet parallele doppelte API-Abrufe.

Counts beziehen sich auf geladene Angebote, aktuell erste Seite mit bis zu 30 Treffern. Bezirksauswahl reduziert die Liste; die Legende zählt weiter alle geladenen Bezirke.

Algorithmen und Darstellung · Code: [DistrictDataService.cs](#) · Z. 22/51/62

Der Weg von der Anmeldung zum konkreten Angebot.

01

Konto / Login

02

Karte + Liste

03

Bezirk wählen

04

Detail + Bilder

05

Abmelden[Lokale Produktdemo öffnen ↗](#)

Vorbereitete Live-Demo

- Backend auf Port 5197; Angular auf 4200.
- Lokaler Entwicklungsmodus mit `npm start`.
- Internet und funktionierender API-Zugang für Angebote.

Wenn die API ausfällt

- Sichtbaren Fehlerzustand ehrlich benennen.
- [Gespeicherte echte Demoansichten](#) zeigen.
- Testausgaben und Fixture-Abläufe erklären.
- Live-Ergebnis und kontrollierten Test getrennt erklären.

Liveprobe am 04.10. bestanden: 28 Angebote, Bad Cannstatt, Detail/Bildwechsel und Logout. Screenshots sind offline verfügbar.

Ergebnisse sind belegt. Ihre Grenzen ebenfalls.

Prüfung am 04.10.2026	Ergebnis	Grenze / offener Punkt
Frontend-Komponenten / HTTP	8 / 8 bestanden	Mockdaten; kein Browser-E2E oder echter Authflow
Frontend-Entwicklungsbuild	Erfolgreich	Initialbundle: 4,43 MB
Frontend-Produktionsbuild	Fehlgeschlagen	1,71 MB überschreiten das 1-MB-Budget
Backendbuild / vorhandene Checks	0 Fehler / Checks bestanden	28 Nullability-Warnungen; Fixtures und Testauth
Echte Desktop-Demoprobe	Bestanden	Live-API HTTP 200; Login → Bezirk → Bilder → Logout

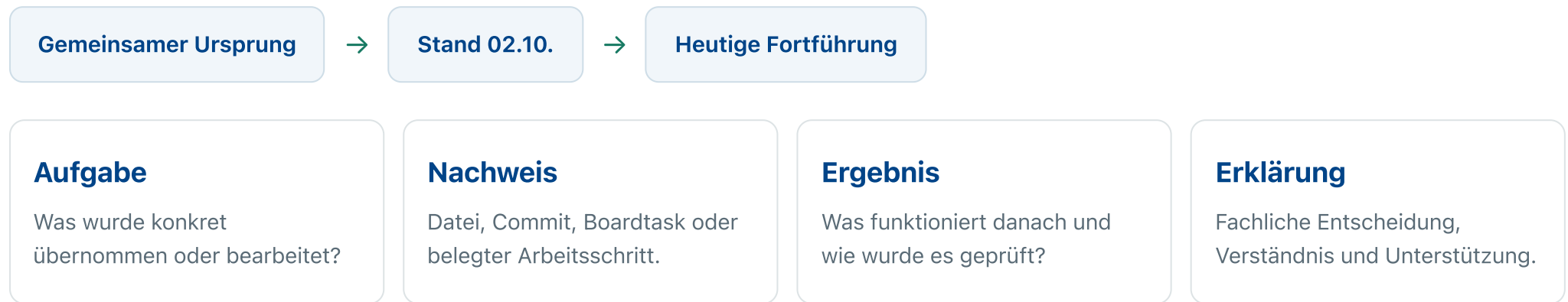
Zusätzlich isoliert geprüft: echte Registrierung, Login und Refreshrotation mit temporärer SQLite. Kein Live-Immobilienabruf.

Reproduzierter Fehler: serverseitige Registrierung akzeptiert ungültige Eingaben. Automatischer Frontend-Refresh bleibt offen. Kartenmarkierung verschwindet beim Rückweg.

Prüfungen und Grenzen · Liveprobe / Sicherung · Code: Frontendtests · Backendchecks · Frontend-Auth · Z. 33

Gemeinsamer Start. Nachvollziehbare Fortführung.

Frühere gemeinsame Arbeit bleibt sichtbar. Der Stand vom 02.10.2026 und spätere Änderungen werden getrennt dokumentiert.



Persönliche Zuordnung einzelner Features ist noch zu bestätigen. Vorhandener Code allein belegt keine Solo-Autorenschaft.

Scrum: Aufgabenstatus, Verantwortlichkeit, geschätzte/tatsächliche Zeit und Retrospektive. Fehlende historische Zeiten bleiben als unbekannt kenntlich.

[Projektboard](#) ➤ [Backlog](#) [Retrospektive](#) [Boardabgleich / Phasen](#)

Der Produktweg steht. Die nächsten Schritte sind klar.

C#, externe API und SQLite sind eingebunden. Wesentliche Datenabläufe sind getestet; der konkrete Desktopablauf mit echter API wurde erfolgreich geprobt.

1 Abgabe nachvollziehbar machen

Aktuellen Code, Startanleitung, Architektur/Algorithmus, Scrum und individuellen Beitrag konsistent abgeben.

2 Technische Korrektheit erhöhen

Produktionsbundle, Backendvalidierung, Frontend-Sitzungsablauf und externe API-Fehlerbehandlung.

3 Bedienung verbessern

Mobile Ansicht, Bezirksreset, Kartenhervorhebung beim Rückweg, Bildfallback und Tastaturzugang.

Technische Fixes werden hier als nächste Schritte genannt, nicht als bereits abgeschlossen. Weiterer Funktionsumfang folgt nach Stabilisierung des vorhandenen Ablaufs.

[Startanleitung](#) [Vollständiger Vortragsleitfaden](#)